AIRBALL

78 Years of Basketball, One Story

COM-480 Data Visualization · EPFL · Spring 2026

Elias Mir · Michael Freeman · Yassine Mamlouk · Aziz Laadhar

Live project → airball website · GitHub → repository

1. Abstract

The NBA has been radically transformed by analytics over the past two decades: the three-point explosion, the death of the mid-range shot, and the rise of pace-and-space offenses. Yet most fans consume the league through box scores and highlight reels, never seeing the macro-level patterns that drive these changes. **Airball** is a four-act interactive data story that makes those structural shifts visible to a non-technical audience.

Our final product is a single-page web application combining scrollytelling narrative (Acts I, IV) with open-ended Gapminder-style exploration (Acts II, III), built on D3.js v7 with a Python pre-processing pipeline. The source tables span NBA, ABA, and BAA history from 1946–2026; after NBA-only and quality filters, the final player table contains **19,833 qualified NBA player-seasons** from **1952–2026**. The site is fully responsive, accessible, and lightweight (no build step, no framework).

2. Idea Evolution: From Proposal to Product

From Milestone 1

The original proposal organized the project around **four questions** — how play style evolved, who the most efficient players of each era were, whether payroll buys wins, and how predictable the draft is. We sketched five candidate visualizations spanning shot charts, era-normalized scatter plots, payroll-vs-wins maps, and draft-class lattices.

What Changed by Milestone 2

After the M1 review, we cut **draft predictability** and **payroll-vs-wins** in favor of depth on the three remaining narratives. Two reasons drove this:

- **Salary data quality.** Inflation-adjusted salaries across 35 years required reconciling three sources with inconsistent name normalization. Time would have come from polish elsewhere.
- **Story coherence.** Pruning gave us a clean four-act dramatic arc: macro trend (I) → individual exploration (II) → fair comparison (III) → team-level story (IV). Each act answers a different question but builds on the same data spine.

From Milestone 2 to Final

The M2 prototype used Chart.js across the board and re-rendered every chart on each user interaction. For M3 we made three structural changes:

1. **Migrated the production charts to D3.js v7** — proper scales, axes, and enter/update/exit selections across all four acts.
2. **Added smooth transitions** — bubbles in Act II now glide between years instead of snapping; radar polygons morph rather than redraw.
3. **Added a fourth narrative beat** — gold championship markers on the dynasty chart, anchoring win-% trends to the trophies that defined each franchise.

3. Design System

The visual identity was set in M2 and held throughout M3.

Element	Choice	Rationale
Background	#080d1a (near-black navy)	Lets the orange accent pop; reduces eye strain; matches sports broadcast aesthetic
Primary accent	#e8621a (NBA-orange)	Direct reference to the basketball; signals 'sports data' without explicit branding
Secondary	#1d6fe8 (cobalt)	High contrast vs orange for player-A/player-B comparisons in Act III
Highlight	#f5c518 (gold)	Reserved exclusively for championship markers — meaningful, sparing use
Headings	Bebas Neue	Tall, condensed sans — sports-broadcast typography
Body	DM Sans	Highly legible at small sizes; pairs cleanly with Bebas
Mono	JetBrains Mono	Used for stat values and meta tags — gives numbers a 'data' texture
Position palette	Distinct hue per position	Maintains semantic colour mapping across acts (PG cyan, SG orange, SF green, PF purple, C red)

The court-line motif in the hero section is a faint structural reference, not decoration — it cues the reader that this is a basketball story before they read a word.

4. The Four Acts: Design Journey

Act I — The Revolution

Goal. Show that the three-point shot went from gimmick to system — a single trend, told dramatically.

Iteration.

1. *M1 sketch:* A static line chart with all annotations drawn at once. Felt like a textbook figure.
2. *M2 prototype:* Added a left-column "story blocks" panel; clicking a block highlighted the corresponding chart region. Better — but the chart re-drew abruptly.
3. *M3 final:* D3 transitions on step change. The orange highlight and shaded area now grow smoothly from 1980 to the chosen year; annotations fade in as you advance. The viewer feels the era unfolding rather than seeing it pre-drawn.

Design decisions. We deliberately limited the chart to a single series. Adding pace or TS% would have diluted the story. The narrative panel does the work of 'what does this mean' so the chart can do only one thing perfectly.

Act II — Era Explorer

Goal. Let any user become an explorer. Play through 45 seasons of NBA history and see eras as visual states.

Iteration.

1. *M1 sketch:* Per-era small multiples (one scatter plot per decade). Discarded — too static, no sense of motion.
2. *M2 prototype:* Bubble chart with a year slider. Worked, but each year flick rebuilt the whole chart, killing the sense of continuity.
3. *M3 final:* D3 enter/update/exit keyed by player. LeBron's bubble glides from his 2010 spot to his 2011 spot. A giant year watermark inside the chart anchors the user in time. The play button at 500 ms/year is fast enough to feel cinematic, slow enough to read.

Design decisions. Encoding: x = Usage Rate, y = True Shooting %, size = minutes per game, colour = position. Two of these (TS% and USG) were chosen because they're era-stable — fair to compare 1985 Magic Johnson to 2024 Luka Don■i■. Only top 30 players per season by minutes played are shown. Search highlights matching bubbles across all years.

Act III — Player vs Player

Goal. Settle 'Jordan or LeBron?' with a fair, era-normalized comparison.

Iteration.

1. *M1 sketch:* Small-multiple bar charts (one per metric, two bars per chart). Visually busy and slow to read.
2. *M2 prototype:* Radar chart with raw stats only. Looked clean but produced misleading conclusions — modern players' TS% will always beat 1970s players regardless of skill.

3. *M3 final:* Added a Normalized / Raw toggle. Normalized scores are 5th–95th percentile rankings computed across all NBA history — they answer 'how dominant was this player relative to their peers' rather than 'what were their raw numbers'. The toggle preserves both perspectives.

Design decisions. Radar charts are often criticized for their distortive area math, but for two-player comparisons with clearly labelled axes they communicate dominance gaps faster than parallel bars. We accepted the trade-off and mitigated it with reference rings at 25/50/75/100.

Act IV — Dynasties

Goal. Let users track a franchise's winning history and feel where the championships cluster.

Iteration.

1. *M1 sketch:* Multi-line chart, one line per team. Looked like a bowl of spaghetti — 30 lines are unreadable.
2. *M2 prototype:* Team selector with a single highlighted line. Clear, but felt thin for the capstone act.
3. *M3 final:* Gold star markers on top of win-% lines at championship seasons. A hover tooltip shows roster highlights. The stars give the chart a punchline: dominance without a ring is just potential.

5. Technical Architecture

Stack

- **D3.js v7** for all four acts (custom transitions, enter/update/exit)
- **Pandas + NumPy** for one-shot data preparation
- **Vanilla CSS** (custom properties, grid, media queries) — no framework
- **GitHub Pages** for hosting

Data Pipeline

scripts/extract_data.py is a one-shot script that:

1. Loads four CSVs (~80 MB combined) from data/, or from the committed archive/ fallback when data/ is absent.
2. NBA-only filter; ABA and BAA seasons excluded.
3. De-duplicates mid-season trades (keeps aggregate rows such as TOT and 2TM).
4. Quality filter: ≥10 games and ≥10 minutes per game.
5. Merges per-game with advanced metrics on (season, player_id).
6. Per act: aggregates, normalizes, and emits one compact JSON to js/.

Output sizes: Act I 1 KB, Act II 204 KB, Act III 877 KB, Act IV 2 KB. Total budget kept under 1.1 MB.

Performance

- **Lazy-loading.** JSONs are fetched per-act on first navigation, not on initial page load. Time-to-interactive drops from ~1.2 s to <300 ms.
- **One render, then update.** D3 root groups are appended once; subsequent updates mutate them, avoiding DOM churn.
- **CSS containment.** Charts use SVG with explicit viewBoxes so the browser can scale them without layout reflow.

Accessibility

- Semantic ARIA labels on all interactive controls.
- :focus-visible outlines for keyboard navigation.
- prefers-reduced-motion respected — all transitions are disabled for users who request it.
- Tested at 360 px width and verified responsive at 540 px and 900 px breakpoints.

6. Challenges and Lessons

Name Normalization

Problem. Player names across CSV files use inconsistent formats (*Magic Johnson* vs *Earvin Johnson Jr.*). Joining on names would have produced silent mismatches.

Solution. Joined on Basketball-Reference's stable player_id slug throughout.

Era-Normalized Comparison

Problem. Comparing raw TS% across eras produces nonsense — the league average has climbed from 52.9% (1980s) to 56.8% (2020s) regardless of individual skill.

Solution. The Act III normalized mode ranks each player's career average against the 5th–95th percentile band of all historical careers. Jokic at 95th percentile *in his era* now reads identical to Jordan at 95th *in his*.

Mid-Season Trades

Problem. Players traded mid-season appear in the raw data once per team plus an aggregate row (TOT, 2TM, or 3TM), triple-counting their stats.

Solution. Detect any season where an aggregate row exists and drop the per-team rows for that player-season.

Performance vs Fidelity in Act II

Problem. Showing all ~450 player-seasons per year produced an unreadable blob.

Solution. Keep the top 30 by minutes played per season. This trims data by 93% and the resulting chart is the league's true competitive core — the players whose minutes shape outcomes.

Smooth Transitions

Problem. The M2 prototype rebuilt the entire chart on each interaction.

Solution. D3's enter/update/exit pattern lets us keep bubbles between frames and tween their attributes, which is what produces the Gapminder-style flow. This was the single biggest user-visible improvement between M2 and M3.

Design vs Information Density

Problem. Several iterations of Act III added secondary metric tables, score deltas, and per-season breakdowns.

Solution. We deleted all of them — the radar plus the two stat columns is enough, and clutter would have hurt the punchline.

7. What We'd Do with More Time

- **Shot-chart overlay** for Act I, showing the spatial migration from mid-range to corner threes (the data exists in the shooting CSV from 1997+ but cleaning it well takes a week).
- **Payroll efficiency** view as a fifth act — winning is interesting but *winning cheaply* is a different and underexplored question.
- **Touch-optimized Act II** — the play/scrub interaction on mobile works but isn't as delightful as on desktop.
- **More dynasties** — we picked six storied franchises but the toggle architecture would extend to all 30 NBA teams trivially.

8. Peer Assessment

Each member contributed meaningfully across multiple areas. The table below summarizes the main ownership for the final submission; several items were reviewed or polished collaboratively after the lead implementation pass.

Area	Elias	Michael	Yassine	Aziz
Data cleaning & pipeline	Support	Lead	Co-lead	Validation
Act I (Revolution)	Co-lead	Review	Lead	Co-lead
Act II (Era Explorer)	Lead	Interaction QA	Data QA	Review
Act III (Player vs Player)	Normalization QA	Lead	Review	Co-lead
Act IV (Dynasties)	Co-lead	Co-lead	Lead	Narrative review
Visual design & CSS	Review	Co-lead	Lead	Accessibility QA
Process book	Co-lead	Review	Methods review	Lead

Area	Elias	Michael	Yassine	Aziz
Screencast	Story review	Timing review	Demo QA	Script/edit lead

Personal Reflections

Elias

This project made clear how much a data story changes between a working chart and a persuasive narrative. I learned to treat annotation, pacing, and visual hierarchy as part of the analysis rather than as decoration added at the end. If we had more time, I would start the narrative storyboard earlier so every act could be tested with outside readers before the final implementation sprint.

Michael

I focused on how the interaction should feel to a non-technical basketball fan. The biggest lesson was that fewer controls often create a stronger experience: Act II became easier to understand once we limited the chart to the top minutes-leaders, and Act III improved when the extra tables were removed. Next time, I would run more structured usability tests on mobile because the project is informative there, but the desktop version is still the richer experience.

Yassine

The data preparation phase showed how fragile sports datasets can be even when they look clean. Mid-season trades, player-name variants, and missing shooting-era fields all created subtle risks that would have changed the story if left unchecked. I learned to prefer stable identifiers and explicit filters over clever joins. With more time, I would add automated validation checks around every exported JSON file.

Aziz

The final milestone taught me how much motion can clarify continuity. Rebuilding the prototype in D3 made the player bubbles and radar shapes feel like persistent objects instead of screenshots, which better matches the story we wanted to tell about eras changing over time. If I repeated the project, I would lock the data contracts earlier so design, interaction, and writing could evolve without reworking the same assumptions.

Process Book · Milestone 3 · May 2026